



Agentic Controller

AI-Powered Migration at Scale on Kubernetes

July 16, 2026

Agenda

1. What is the Agentic Controller?
2. CRD architecture – the building blocks
3. Fitting into the Konveyor ecosystem – Hub & UI
4. End-to-end flow
5. The harness – orchestrating agent work
6. Demo – harness & controller in action
7. What's next

What is the Agentic Controller?

A **Kubernetes controller** for managing AI agent workloads in Konveyor

- Defines domain-agnostic CRDs for composing and executing agent tasks
- Leverages **Agent Sandbox** (k8s SIG) for isolated execution
- Decoupled from Hub – portable, GitOps-friendly, `kubectl`-native
- Born from the tackle2-addon-kai POC, rebuilt as proper k8s primitives

The controller validates references, creates Sandboxes, and drives lifecycle – but it has **no domain-specific intelligence**. It sees desired state and makes it happen.

CRD Architecture – Templates

Definition resources declare what's available (inert, no execution):

CRD	Purpose
SkillCard	Individual skill or rule (OCI artifact via skillimage)
SkillCollection	Group of related skills
LLMProvider	LLM endpoint, credentials, model catalog
Agent	Template: image, providers, skills, prompt, parameters
AgentPlaybook	Ordered sequence of stages, each referencing an Agent

CRD Architecture – Execution

Execution resources trigger work (analogous to Tekton TaskRun):

CRD	Purpose
AgentRun	Concrete invocation of an Agent with specific values
AgentPlaybookRun	Runs a playbook – creates sequential AgentRuns

Agent vs AgentRun – the key distinction:

- **Agent** = what's available (template)
- **AgentRun** = what's selected (concrete values, triggers a Sandbox)

Parameters are **opaque** – the controller injects them but never interprets their meaning.

Key Design Principles

1. **Controller is domain-agnostic** – validates, creates Sandboxes, tracks lifecycle – but never calls Hub, Backstage, or interprets parameter values
2. **Agent Sandbox is a hard dependency** – no bare-Pod fallback
3. **Git credentials stay in the harness** – agents cannot push
4. **UI connects to agent pods directly** for ACP streaming
5. **Hub provides curated REST endpoints** – single API surface for the UI

Fitting into Konveyor – Hub Integration

Hub exposes **purpose-built REST endpoints** for all agent resources:

/hub/agents	/hub/agentplaybooks
/hub/agentruns	/hub/agentplaybookruns
/hub/skillcards	/hub/skillcollections
/hub/llmproviders	

AgentRun creation is a smart endpoint – UI sends minimal request (agent name, app ID, model, instructions); Hub resolves git URL, credentials, branch name, and builds the full CR.

Hub also **proxies ACP WebSocket** to agent pods for streaming.

Fitting into Konveyor – The UI

- UI talks **only to Hub** – no k8s API, no raw CR formats
- Real-time agent streaming via **ACP over WebSocket** (proxied through Hub)
- **Human-in-the-loop**: approve permissions, redirect, cancel – all via ACP
- Multiple clients can connect; `session/load` replays full history on join

```
UI  —REST—>  Hub  —controller-runtime—>  k8s API
UI  —WebSocket—>  Hub  —proxy—>  Agent Pod (ACP :4000)
```


End-to-End Flow

UI → Hub: "Run agent X on application Y"
Hub: resolve git URL, credentials, branch
Hub → k8s API: create AgentRun CR

Controller: reconcile AgentRun
→ create Sandbox CR

Sandbox: start pod with harness + agent runtime
Harness: clone repo, configure workspace
Harness: launch agent (e.g. goose serve)
Agent: detect → plan → execute → verify → fix-loop
Harness: commit work, push to target branch

Controller: watch Sandbox status → update AgentRun status
UI: stream agent output via ACP WebSocket

The Harness

The **Go binary entrypoint** in every agent base image:

- **Git lifecycle** – clone, configure remotes, incremental commits, push on exit
- **Agent launch** – starts the agent runtime (e.g. `goose serve`)
- **Credential isolation** – agent gets a workspace remote with no push credentials
- **Handoff files** – writes `.konveyor/handoff.md` for cross-stage continuity

Workflow: **detect** → **plan** → **execute** → **verify** → **fix-loop**

Artifacts: session state, handoff markdown, verification reports, execution logs, metrics

Multi-Stage Playbooks

AgentPlaybook defines ordered stages, each with its own Agent:

Stage	Agent	What it does
analyze	code-analyzer	Detect migration targets and produce a plan
migrate	code-migrator	Execute the migration plan
verify	test-runner	Run tests and verify correctness

Each stage runs in a **fresh Sandbox**, sharing work via **git commits** on a shared branch. The playbook guide is injected as `KONVEYOR_PLAYBOOK_INSTRUCTIONS`.

Demo – Harness & Controller

PR #33: Harness init – the foundation

- Migration harness CLI: `init`, `run`, `status`
- Full workflow: detect → plan → execute → verify → fix-loop
- ACP session management (WebSocket to Goose)
- Git credential handling and workspace management
- Recipes and skill bundles for migration scenarios
- Container image: `agent-base-goose-java`
- Comprehensive test coverage

Hiteshwari & Savitha will walk through the implementation.

What's Next

Landing now: PRs #33 (harness), #35 (client stack), #36 (playbook controllers)

Roadmap:

Area	What
OpenShell	Kernel-level isolation and credential protection
Tekton custom task	AgentRun as a Tekton task – parallelism, retries, signing
SkillCard resolution	Build OCI artifacts from git repos and inline markdown
Memory service	Persistent knowledge base via MCP across runs
Production images	Harness, runtime, and language toolchain layers



Demo Time

Harness & Controller Implementation

Hiteshwari & Savitha



Questions?

github.com/konveyor/agent-controller